Markov Decision Process-Based Artificial Intelligence With Card-Playing Strategy and Free-Playing Right Exploration for Four-Player Card Game *Big2*

Lien-Wu Chen, *Senior Member, IEEE*, and Yiou-Rwong Lu

***Abstract*—The popular East Asian card game *Big2* involves rules that do not allow players to view each other's hand cards, making artificial intelligence face challenges in performing well in this game. Based on Markov decision processes (MDPs) that can handle partially observable and stochastic information, we design the Big2MDP framework to explore card-playing strategies that minimize losing risks while maximizing scoring opportunities for the *Big2* game. According to our review of relevant research, this is the first *Big2* artificial intelligence framework with the following features: first, the ability to simultaneously consider scoring and losing points to make the best winning decisions with minimal losing risk, second, the capability to predict multiple opponents' actions to optimize the decision-making, and third, the adaptability to compete for the free-playing right to change card combinations at the essential moment. We implement a system of four-player card game *Big2* on the Android platform to validate the feasibility and effectiveness of Big2MDP. Experimental results show that Big2MDP outperforms existing artificial intelligence methods, achieving the highest win rate and the least number of losing points as competing against both computer and human players in *Big2* games.**

***Index Terms*—Artificial intelligence, card game, machine learning, Markov decision process (MDP), mobile device.**

I. Introduction

BIG2 is a popular card game in East Asia, also known as "Da Lao Er" (meaning "big two" due to the highest number being two) and "*Chu Da Di*" (a game historically played by farmers during their leisure time). It originates in the coastal areas of China in the late Ming dynasty and has since evolved into other similar card games, such as "*Dou Di Zhu*" and "*Da Fu Hao*." The game is played with a standard deck of 52 cards (excluding two jokers) and is typically played by four players. At the beginning of the game, all cards are evenly distributed among the players. Players take turns playing their card sets, and each card set played must match the previous player's card set in both type and be of a higher value. If a player cannot or chooses not to play a card set, they can pass their turns. When three players in a row pass, the last player to play becomes the "lead" and can play any card set he/she wishes (i.e., free-playing right). The game ends when a player has played all of his/her cards. Cards are ranked from 3 as the lowest to 2 as the highest, and card suits are ranked from smallest to largest as Club, Diamond, Heart, and Spade. The game includes six types of card combinations: Single, Pair, Full House, Straight, Four of a Kind, and Flush Straight (i.e., six card sets). Only four of a Kind and Flush Straight can be played at any time and can override any other card type. All other card combinations must match the previous player's card combination type. In *Big2* game rules, each card is worth the same number of points as losing, whereas the remaining card with the number 2 makes total losing points doubled, and if a player holds equal to or more than 10 cards, the total losing points are also doubled (see Section V for a scoring example) [1].

In recent years, artificial intelligence has been rapidly advancing, and since AlphaGo's victory against the world champion of *Go* [17], there have been high expectations for its application in the gaming domain. However, there is a crucial difference between games, such as *Go* and *Big2*, a poker card game: in *Big2*, players cannot view each other's hand cards, which creates an imperfect information setting with stochasticity and makes it challenging for artificial intelligence to dominate. Although numerous algorithms have attempted to enable artificial intelligence to operate in imperfect information scenarios, many of them employ Determinization [2], a method of classifying uncertain elements into the same category as known elements to significantly reduce complexity. While this approach allows artificial intelligence to play the game, it falls short of giving them the dominating power in imperfect-information games.

Various algorithms have been used in games, including Monte Carlo tree search (MCTS) [3], [4], [5], [6], [7], information set with MCTS (ISMCTS) [2], [8], and AIs for imperfect information games [9], [10], [11], [12], [13], [14], [15], [16]. However, MCTS relies on the accuracy of its simulations, and in imperfect information games, crucial information affecting the outcome is often lacking. As a result, the simulated results may deviate

Manuscript received 24 August 2023; revised 22 February 2024 and 20 May 2024; accepted 28 June 2024. Date of publication 8 July 2024; date of current version 18 June 2025. This work was supported in part by the Ministry of Science and Technology, Taiwan, under Grant 109-2221-E-035-062-MY3, Grant 112-2221-E-035-032, and Grant 113-2221-E-035-073-MY3. Recommended by Associate Editor J. LIU. *(Corresponding author: Lien-Wu Chen.)*

The authors are with the Department of Information Engineering, Feng Chia University, Taichung 407, Taiwan (e-mail: lwuchen@fcu.edu.tw).

Digital Object Identifier 10.1109/TG.2024.3424431

TABLE I
COMPARISON OF PRIOR WORKS (AGENTS FOR VARIOUS GAMES [17], [18], [19], [20], [21], [22], [23], [26], [27], [28], [30], [31], [32], [33] AND AGENTS FOR BIG2 [34], [35], [36]) AND OUR BIG2MDP FRAMEWORK

Feature	Imperfect infomation consideration	Losing point minimization	Global reward maximization	Multi-opponent prediction	Card-playing prediction	Free-playing preemption
[28], [29]			√			
[19], [33]	√					
[35], [36]	√			√		
[18], [20], [21], [23], [24], [32]	√	√				
[34]	√	√	√			
[22], [27], [31]	√	√	√	√		
[37]	√	√	√	√	√	
Our Framework	√	√	√	√	√	√

from real-world situations, making the computed card-playing solutions less reliable than those in perfect information games.

In addition, well-known game AIs have been designed that beat human players, including the *Go* agent of AlphaGo [17], agents for card games "*Magic: the Gathering*" [18] and "*Hearthstone*" [19], the agent in "*StarCraft*" that analyzes opponent behavior for decision-making [20], agents for card games "*Bridge*" [21] and "*Texas Hold'em*" [22], the agent for decision-making in fast-paced fighting games [23], the agent for the two-player game of "*Stratego*" [24], and agents using minimax, expectimax, and MCTS for the single-player puzzle game of "2048" [25]. However, achieving the same level of performance in imperfect information games like *Big2* is challenging for aforementioned game agents. It is because the right to change card combinations (i.e., free-playing right), which allows us to play any card combination has to be competed, where our next move is not blocked by opponents. In addition, in the *Big2* game, the most important goal is to be the first player who plays the last card (i.e., the winner). The number of losing points is only considered as dealing a hand with a very low chance to win. Furthermore, the goal may change with different remaining cards in hand (e.g., fastest path to win with high-ranking cards, minimizing losing points with low-ranking cards, competing for the free-playing right to cover low-ranking cards, holding free-playing rights until the last card is played, etc.), where each goal has to be achieved under its corresponding condition.

In particular, Markov decision process (MDP) algorithms have been adopted in game agents [26], along with extended methods for resource sharing in MDPs [27], simplified MDP computation [28], agent for the *Hide-and-Seek* game using partially observable MDPs [29], agent for *Mahjong* using multiple MDPs [30], agent for the game "*Blood Bowl*" using MDPs [31], agent for the game "*Snakes and Ladders*" using MDPs [32], and the implementation of MDPs in zero-sum games [33]. Furthermore, existing *Big2* AI algorithms have been proposed including the rule-based AI for decision-making [34], the AI using reinforcement learning [35], and the AI using information set for predicting opponent's moves [36]. However, existing *Big2* AIs do not consider card-playing strategies of competing for free-playing rights to cover low-ranking cards and/or hold the right until the last card is played.

In this study, we design the Big2MDP framework to explore card-playing strategies that minimize losing risks while maximizing scoring opportunities based on MDPs for the card game *Big2*. Card-playing route prediction and free-playing right competition for the *Big2* game are explored in the design of the Big2MDP framework. Table I shows a comparison of the features provided by existing artificial intelligence algorithms (discussed in Section II) with ours. Our framework offers the most complete solution to the imperfect information *Big2* game according to whether it can 1) handle the issue of not seeing opponents' hand cards, 2) deal with the minimization of losing points, 3) consider the overall game end when calculating rewards, 4) predict next moves of multiple opponents, 5) predict opponents' action routes, and 6) compete for the right to change card combinations as necessary.

The contributions of our framework are four-fold. First, it can calculate the rewards obtained from different winning moves in the card game *Big2* and find the action path that maximizes the rewards. Second, it can identify whether victory becomes difficult and discard routes with high rewards but lower chances of winning to minimize losses. Third, it can predict opponents' possible actions based on limited information in the game and historical game data. Finally, it can evaluate the strengths and weaknesses of card combinations in hand and compete for the free-playing right to increase the winning rate and score. Furthermore, an android-based system for four-player card game *Big2* is implemented to validate the feasibility and effectiveness of Big2MDP. Experimental results show that Big2MDP outperforms existing artificial intelligence methods, achieving the highest win rate and the least number of losing points when competing against both computer and human players in *Big2* games.

The rest of this article is organized as follows. Section II discusses existing works. Section III defines the card-playing prediction and free-playing contention problem for four-player card game *Big2*. Section IV presents our framework based on MDPs. Section V demonstrates the system implementation of our framework. Experimental results are shown in Section VI. Finally, Section VII concludes this article.

II. RELATED WORK

In this section, we discuss existing works in agents for *Big2* [34], [35], [36] and in agents that use MDPs for other games [26], [27], [28], [30], [31], [32], [33], where MDPs have been used to enable artificial intelligence to tackle complex problems. Moghadasi et al. [26] presented an MDP-based AI designed for playing 3-D soccer games, which involve uncertainty factors compared to common static games, significantly

affecting the AI's decision-making. In particular, the soccer field is partitioned into a 3×3 grid based on players' positions to simplify the complex coordinate system in 3-D games. Wu and Feng [27] introduced a self-learning approach for MDPs, employing Kullback–Leibler divergence to assess the similarity between states and identify historical states that resemble the current state. If similar states are found, past data are reused to save redundant computations; otherwise, MCTS is utilized for random simulation, reducing the search time for each iteration. To handle problems with a large number of actions, Garcia-Hernandez et al. [28] calculated the cost and reward of actions and then compared them with other actions. Actions with similar results are merged into one, reducing the complexity of the problem, which can reduce the computation time for deciding which card set to be played in the *Big2* game.

Kurita and Hoki [30] presented an AI for playing *Mahjong* using MDPs. While an MDP can efficiently handle single-reward problems, it may overly emphasize specific directions and overlook the impact of other factors in complex games. This *Mahjong* AI combines multiple MDPs to concentrate different tasks: one for acquiring scores, another for discarding tiles, one for calculating winning routes, and an additional one for making crucial decisions, such as when to declare "Ready" to win, which occurs only once per game but is highly significant. By timely adjusting the reward weights of different MDPs under different circumstances, the AI achieves the highest winning rate among existing methods. Similar to *Mahjong*, *Big2* is a multitask game to seek the highest score, reducing losses, achieving the fastest victory, and competing for free playing rights.

Humeau et al. [31] utilized MDPs to play the tabletop game Blood Bowl. Although *Blood Bowl* is a one-on-one game with transparent information, it combines American football with brutal combat, resulting in a game that can span up to 10×50 rounds. The complex gameplay attracts many players but also makes it challenging for AI to compete. The study converts actions into a probability problem to estimate the most advantageous move within a single round.

In [32], the AI played *Snakes and Ladders* to compare two MDP strategies: finding the shortest path to victory and maximizing the number of victories. While longer paths usually incur no additional costs in games, such as *Go*, in *Big2*, for instance, one can be easily blocked by opponents' high-ranking cards, disrupting the planned route to victory. Thus, the evaluation of longer paths needs to consider the additional risk of being blocked in the *Big2* game.

Zhu and Zhao [33] dealt with a two-player zero-sum game AI, where one player's gain comes from the other player's loss. *Big2* is an example of this type of game. In such games, the objective for calculating rewards is different from typical scoring games. In *Big2*, the more remaining cards a player has when the game ends, the more points they lose, while the winner gains more points. In addition, when a player cannot beat an opponent's card, they must skip their turn. Utilizing this rule, an alternative method for increasing rewards can be computed.

Fernando and Tai [34] is a rule-based *Big2* artificial intelligence. It predefines the value of each poker card, equivalent to their suit and numerical rank in the game. If multiple cards form a combination, their values are further weighted based on the type of combination. During gameplay, it determines its actions based on whether it has the right to play. If it has the right to play, it decides whether to play high-value cards to attempt to retain the continuous playing right or to play low-value cards to ensure opportunities for playing other low-value cards in the hand. If it does not have the right to play, it prioritizes playing low-value cards and selects PASS if no cards are playable. Since it only considers the cards in hand, this AI requires minimal computational time. However, its playing strategy tends to prioritize playing cards from small to large and leaving more cards in hand. As a result, it can be challenged by human players in real game scenarios. To achieve victory in actual gameplay, more flexible rules are needed.

Charlesworth [35] employed the proximal policy optimization (PPO) reinforcement learning algorithm. It uses the known game state information as input and estimate the winning probability of their own hand. PPO utilizes the trust region method to search for the highest reward within a certain range and gradually expands the search scope. The difference between PPO and the general trust region policy optimization lies in the incorporation of Kullback–Leibler divergence to balance extreme sample discrepancies, preventing extreme scenarios from disproportionately influencing the overall computation. This artificial intelligence's performance in playing against human players is comparable. Several reasons are discussed for the minor performance gap, including the highly complex possibilities in each turn of *Big2*, making it challenging for deep learning models to rely solely on known information on the table to make accurate decisions. Moreover, the AI did not predict and evaluate future states, as it only calculated immediate outcomes.

Chen and Lu [36] designed a *Big2* artificial intelligence that combines information set with MCTS. This integration allows the AI to classify game states based on public information, searching for similar past game states according to categorized features, in order to guess the opponent's hand and predict their subsequent card play. Simultaneously, it employs counterfactual regret minimization to dynamically adjust regret values based on the prediction results, assigning different regret values to the same card in different contexts, thus determining the most suitable action for the current situation. Although the AI achieves a 40% accuracy in predicting the hand of the first opponent, a mere 10% accuracy is for the remaining opponents. This low prediction accuracy is also reflected in its win rate, as the AI's performance in playing against human players is not impressive. In particular, the card-playing strategies of competing for free-playing rights are not considered to cover low-ranking cards or hold the right until the last card is played.

III. System Model

Fig. 1 shows the system architecture of Big2MDP, where an Android-based system for four-player card game *Big2* is implemented for the research (discussed in Section V). Following the rules of the *Big2* card game, each game can accommodate up to four players. In cases where the number of players is less than four, the game creator can choose to include an MDP AI as one

Fig. 1. System architecture of Big2MDP.

of the players or wait for other players to join through game search.

Regardless of whether an MDP AI is included or not, during the *Big2* game, after any player makes a move, the result is transmitted to the Big2MDP server for verification. Once the server confirms that the executed action complies with the rules of the *Big2* game, it sends the result to other players in the same game. Historical data are stored at the end of each game to facilitate learning for the designed MDP AI in future games. If at least one AI opponent is present in the game, the Big2MDP server performs MDP decisions upon receiving the actions from the preceding player at the end of his/her turn. Once the MDP decisions are complete, the subsequent move is sent to other players in the same game after passing the Big2MDP server's verification.

In order to design an MDP-based AI capable of playing against human players in the imperfect information setting of the *Big2* game, it is essential to enable the AI to compute solely based on limited information and utilize predictions to compensate for missing data. In addition to aiming for high rewards and win rates, due to the high randomness in the *Big2* game, there may be situations where the AI is dealt a hand with very low chances of winning. Hence, it is crucial to consider how to minimize losses in such scenarios. Lastly, we leverage the free-playing right to change card combinations of the *Big2* game to enhance control over the game and prevent opponents from disrupting the originally planned sequence of moves. The goal is to achieve high winning rates while minimizing losing points by addressing the following research issues.

1) *Wining reward maximization:* How to calculate the rewards obtained from different winning moves in the card game *Big2* and find the action path that maximizes the rewards?
2) *Losing point minimization:* How to identify the difficult situations to win and abandon high-reward but low-probability-of-winning routes for minimizing the losses as losing the game?
3) *Card-playing route prediction:* How to predict the possible action routes of multiple opponents based on limited public information in the game and historical game data?
4) *Free-playing right contention:* How to analyze the strengths and weaknesses of each card combination in hand and content for gaining control of the right to change card combinations at the essential moment?

IV. Big2MDP Framework

MDPs are used to solve problems of uncertain search, where the actions taken do not lead to fixed states, which traditional search algorithms cannot find an optimal solution. An MDP calculates the expected rewards for different states, then determines the expected probabilities of transitioning into various states based on the actions taken, and analyzes which actions can achieve the maximum expected reward.

An MDP allows for the flexibility to set different decision criteria within the rules to assist in problem-solving. In a particular case, multiple MDPs need to be combined to find the optimal solution [30]. *Big2* is a game that requires such an approach for seeking the highest score, reducing losses, achieving the fastest victory, and competing for the free-playing right. In the following subsections, we design a series of MDPs based on different goals, which result in a *Big2* AI that can achieve high winning rates while minimizing losing points.

A. MDP 1.0—Highest Scoring Point Decision

Among various MDP strategies, one of the most common and straightforward approaches is to estimate scores and select the highest-scoring solution. In the context of *Big2*, this means estimating the final score for each state and using it as a reward. In a four-player game of *Big2*, let p_0, p_1, p_2, and p_3 represent the players, with the AI player currently making decisions denoted as p_0 and the other players following in the subsequent playing order as p_1, p_2, and p_3. When player p_0 takes any action a in the state s, it will transition to a new state s' with a probability $P_a(s, s')$ and receive a reward $R_a(s, s')$. In *Big2*, each state s corresponds to a single round, meaning each player takes its respective action a in s. The current state is referred to as s_0, and within s_0, the previous round's actions are recorded according to the sequence of actions taken

$$s_0 = \{(p_0, a_0), (p_1, a_1), (p_2, a_2), (p_3, a_3)\}. \tag{1}$$

During the decision-making process, states other than the current state will only consider the actions of the opponents, while the actions of player p_0 are represented by the symbol X as follows, indicating any valid action according to the game's rules:

$$s_1 = \{(p_0, X), (p_1, a_1), (p_2, a_2), (p_3, a_3)\} \tag{2}$$

where the usage of X aims to reduce the number of states with minor differences, allowing any action that has occurred in a particular state to be shared among multiple actions, thereby avoiding the need to create separate states for each action.

In Big2MDP, action a refers to a player p_0 performing either a play or a pass action (i.e., *Play* or *Pass*). *Pass* indicates that a player chooses not to play any card, while *Play* includes all

remaining actions creating a valid card combination from the player's hand and playing it. Each card combination played by player p_0 is considered an individual action a (e.g., playing a pair of Club 2 and Heart 2 is denoted as action $a_{\text{Pair}(c2,h2)}$). Note that according to the *Big2* rules, a player can only play a card combination of the same type as the last played combination if it is not in the leading position. Therefore, actions that are impossible to play are not included in the action set.

The probability $P_a(s, s')$ is the probability of transitioning from state s to state s' after taking action a (regardless of victory or defeat). The probability of an action leading to different states may vary, but the sum of probabilities for all possible states resulting from the same action is 100%. In other words, an MDP does not take into account the occurrence of past states in the probability calculation. The probability is calculated as

$$P_a(s, s') = \frac{N(s, a, s')}{N(s, s')} \tag{3}$$

where $N(s, a, s')$ is the number of times that state s transitions to new state s' through action a and $N(s, s')$ is the total number of transitions from state s to new state s'. In other words, $P_a(s, s')$ is the percentage of occurrences for this particular transition.

In particular, the reward of state s is determined by the next state s', and the reward of state s' is determined by the subsequent state s'', until a terminal state (i.e., the end of the game) is reached. In this way, the entire MDP forms a converging reward formula. Once all the state values are calculated, the AI focuses on the state s' with the highest state value V and selects the action a that has the highest action value Q in reaching state s'.

In *Big2*, only the winner can earn points, while the other three players lose points. Therefore, only the scenarios where a victory is achieved are considered, and we take the maximum value for the expected action value in MDP 1.0

$$Q_a^{1.0}(s, s_i) = \max_{s_i \in C(s)} [P_a^{\text{win}}(s, s_i) \times R_a^{\text{win}}(s, s_i)] \tag{4}$$

where $C(s)$ is the set of all possible new states from state s, s_i is the ith state in $C(s)$, and $P_a^{\text{win}}(s, s_i)$ and $R_a^{\text{win}}(s, s_i)$ are the probability of transitioning and the score obtained after transitioning, respectively, from state s to a new state s_i where a victory is achieved after taking action a. The expected values of card sets held by a player with cards in hand are used to select the card set played by the AI player, which are changing with the number of hand cards [36]. The method used for self-learning is focusing on Q-value updates, where expectimax search is used to maximize the expected reward.

To determine the optimal path, we calculate the value of each possible state s based on the action values, where the expected action value $Q_a^{1.0}(s, s')$ to represent the reward obtained by selecting action a in state s. Then, we select the maximum action value Q as the state value V as follows:

$$V^{1.0}(s, s') = \max_{a_i \in M(s')} Q_{a_i}^{1.0}(s, s') \tag{5}$$

where $M(s')$ is the set of all possible actions in state s', a_i belongs to $M(s')$, and $Q_{a_i}^{1.0}(s, s')$ is the expected action value obtained by performing action a_i in state s'.

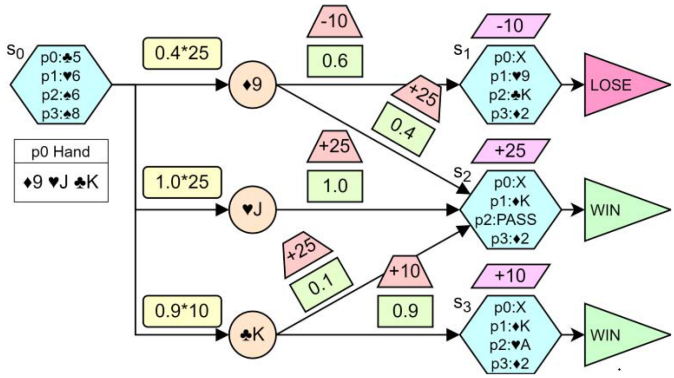


Fig. 2. Example of reward estimation in MDP 1.0.

In MDP 1.0, the reward $R_a(s, s')$ is determined based on the highest score and the state value V is used as the reward

$$R_a^{1.0}(s, s') = V^{1.0}(s, s') \tag{6}$$

where action a represents the possible actions in state s, and the state s' is the new state after executing action a.

Fig. 2 shows an example of the *Big2* game in MDP 1.0, where s_0 is the current state. States s_1 to s_3 are the final states where rewards can be obtained. In the current state s_0, player p_0 played Club 5, and other players p_1, p_2, and p_3 played Heart 6, Spade 6, and Spade 8, respectively. Player p_0 has three cards left in hand: Diamond 9, Heart J, and Club K. This results in three possible actions: $a_{\text{Single}(d9)}$, $a_{\text{Single}(hJ)}$, and $a_{\text{Single}(cK)}$. For simplicity, assume that these actions lead to states s_1, s_2, and s_3, respectively. Among these, state s_1 results in failure (i.e., a score of -10), while the other two states result in victory (i.e., scores of +25 and +10).

Next, assume that the AI calculates the probabilities for each action a based on the formula for $P_a(s, s')$. The action $a_{\text{Single}(d9)}$ has a 0.6 probability of transitioning to state s_1 and a 0.4 probability of transitioning to state s_2. The action $a_{\text{Single}(hJ)}$ has a probability of 1.0 to transition to state s_2, and the action $a_{\text{Single}(cK)}$ has a 0.1 probability of transitioning to state s_2 and a 0.9 probability of transitioning to state s_3. The action values are $Q_{a_{\text{Single}(d9)}}^{1.0}(s_0, s')$ and $Q_{a_{\text{Single}(cK)}}^{1.0}(s_0, s')$ for actions $a_{\text{Single}(d9)}$ and $a_{\text{Single}(cK)}$, respectively. Although action $a_{\text{Single}(d9)}$ has two different scores, one of them, $R_{a_{\text{Single}(d9)}}^{1.0}(s_0, s_1)$, is negative, meaning that this route yields no point. Therefore, $Q_{a_{\text{Single}(d9)}}^{1.0}(s_0, s')$ will ignore this route's score. On the other hand, action $a_{\text{Single}(cK)}$ has two positive scores: $R_{a_{\text{Single}(cK)}}^{1.0}(s_0, s_2) = 0.1 \times 25 = 2.5$ and $R_{a_{\text{Single}(cK)}}^{1.0}(s_0, s_3) = 0.9 \times 10 = 9.0$. Only the highest reward is selected for $Q^{1.0}$ (i.e., $R_{a_{\text{Single}(cK)}}^{1.0}(s_0, s_3) = 9.0$) as the action value $Q_{a_{\text{Single}(cK)}}^{1.0}(s_0, s')$.

B. MDP 2.0—Lowest Losing Point Decision

In MDP 1.0, we only consider the highest score, which leads it to weight the chance of a big win as worth risking the loss to obtain the maximum score in ideal situations. However, in the *Big2* game, it may not always be possible to successfully play the expected card sets in succession. Focusing solely on

forming large card sets (e.g., full house, straight, etc.) might even prevent us from playing the small card sets currently allowed (e.g., single, pair, etc.). One approach to deal with the issue is to consider both the highest score and the lowest loss simultaneously. In addition to considering our own score, we also take into account how to minimize the losing points.

By reducing the priority of the large card sets according to a certain proportion, the AI can consider other smaller card sets that do not have such high priority. In addition, by playing these smaller card sets, the AI can minimize the points lost when losing a game. Therefore, we design MDP 2.0 and extend the reward $R_a(s, s')$ to the average score lost when transitioning from state s to s' after losing the game as

$$Q_a^{2.0}(s, s_i) = \begin{matrix} \max_{s_i \in C(s)} [P_a^{\text{win}}(s, s_i) \times R_a^{\text{win}}(s, s_i) \\ + P_a^{\text{lose}}(s, s_i) \times R_a^{\text{lose}}(s, s_i)] \end{matrix}, \quad (7)$$

where $R_a^{\text{lose}}(s, s_i)$ is the average score lost when transitioning from state s to the new state s_i after losing the game and $P_a^{\text{lose}}(s, s_i)$ is the probability of losing the game after taking action a.

However, only modifying the scores in Q would cause the AI to adopt a conservative strategy throughout the entire game, which does not select card-playing paths with high scores (even with many high-ranking cards in hand). Therefore, we adopt an end-game condition, denoted as S_{end}

$$S_{\text{end}} = \begin{cases} \text{True}, & \text{if } |h_1| \le E_\alpha^{\text{end}} \vee |h_2| \le E_\alpha^{\text{end}} \\ & \vee |h_3| \le E_\alpha^{\text{end}} \vee |g_0| \le E_\beta^{\text{end}} \\ & \vee l \ge E_\gamma^{\text{end}} \\ \text{False}, & \text{otherwise} \end{cases} \quad (8)$$

where h_i is the hand of player p_i in the current state and $|h_i|$ is the number of cards in h_i. The conditions $|h_1| \le E_\alpha^{\text{end}} \vee |h_2| \le E_\alpha^{\text{end}} \vee |h_3| \le E_\alpha^{\text{end}}$ indicate that in state s, the hand of any opponent (i.e., p_1, p_2, or p_3) contains fewer or equal to E_α^{end} cards. g_0 represents the card sets that can be formed from the hand of player p_0 in the current state, excluding the card sets of Single. $|g_0|$ is the number of card sets in g_0, and the condition $|g_0| \le E_\beta^{\text{end}}$ means that the number of card sets that player p_0 can form with its current hand is less than or equal to E_β^{end}. The condition $l \ge E_\gamma^{\text{end}}$ indicates that the total score of the cards on the table (i.e., Table-Card Level) in the current state is greater than or equal to E_γ^{end} levels. The scores are assigned based on the numerical values of the cards: 3, 4,..., 10, J, Q, K, A, 2, with values 1, 2,..., 8, 9, 10, 11, 12, 13, respectively. In addition, each suit (i.e., Club, Diamond, Heart, and Spade) is assigned extra scores of 1, 2, 3, 4, based on their respective ranks. The scores are grouped into levels, with each level covering a range of 10 points. For example, if the total score is 303 points, $l = 30$ levels. S_{end} will be True if any of those conditions is satisfied; otherwise, it will be False. The exact values of E_α^{end}, E_β^{end}, and E_γ^{end} will be specified based on the experiments.

When S_{end} is True, the AI starts checking whether there exists a positive state value V for state s. If there is any positive state value, it indicates that there is still a chance of winning. In contrast, if all state values are negative, the goal is changed to reducing the losses. The decision-making strategy will switch between aggressive and conservative based on the evaluation of S_{end}. Therefore, in MDP 2.0, the state value V takes the following forms depending on different states:

$$V^{2.0}(s) = \begin{cases} \max_{s_i \in C(s)} Q_a^{2.0}(s, s_i), & \text{if } S_{\text{end}} = \text{True} \\ & \wedge\, Q_a^{2.0}(s, s_i) \le 0 \ \ \forall s_i \\ \max_{s_i \in C(s)} Q_a^{1.0}(s, s_i), & \text{otherwise.} \end{cases} \quad (9)$$

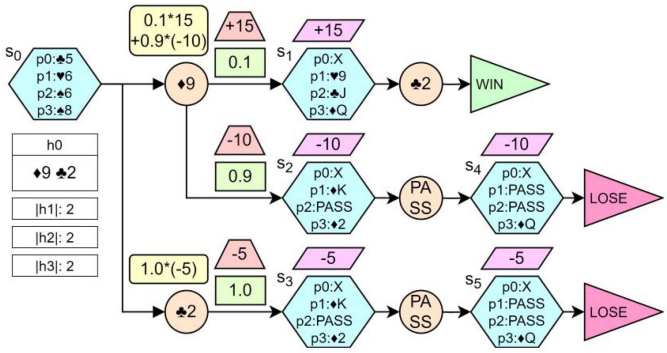


Fig. 3. Example of losing point minimization in MDP 2.0.

Fig. 3 shows an example of the *Big2* game in MDP 2.0. The current state, s_0, is when player p_0 plays the Club 5, and other players, p_1, p_2, and p_3, play the Heart 6, Spade 6, and Spade 8, respectively. Player p_0 has two cards left: Diamond 9 and Club 2, forming two possible actions: $a_{\text{Single}(d9)}$ and $a_{\text{Single}(c2)}$. Since the other three players all have two cards in their hands, the S_{end} for the current state is True, and $Q_a^{2.0}$ will be used to calculate the state values. For action $a_{\text{Single}(d9)}$, it can lead to two states: s_1, where no one plays a card higher than Club 2, allowing player p_0 to win by playing the last card with an expected $R_{a_{\text{Single}(d9)}}^{\text{win}}(s_1, s_{\text{win}}) = 5 \times 3 = 15$ (where three cards left in the hands of other three players and each card gains 5 points); and s_2, where player p_3 plays Diamond 2, and p_0's Club 2 cannot surpass it, resulting in having to choose PASS in the next round and becoming state s_4. In state s_4, player p_3 plays his/her last card, declaring victory, causing player p_0 to lose the game. At that moment, p_0 still has Club 2 in hand, and according to the rule, losing a card with the number 2 multiplies the loss by 2, so $R_{a_{\text{Single}(d9)}}^{\text{lose}}(s_2, s_4) = -5 \times 2 = -10$.

Assuming the probability of $a_{\text{Single}(d9)}$ reaching state s_1 is 0.1 and reaching state s_2 is 0.9, then the action value $Q_{a_{\text{Single}(d9)}}^{2.0}(s_0, s_i) = (0.1 \times 15) + (0.9 \times -10) = -7.5$. The other action, $a_{\text{Single}(c2)}$, will only lead to state s_3, where the actions of other players are the same as in state s_2. Similarly, due to player p_3 playing his/her last card and declaring victory, player p_0 loses the game, but since p_0 has Diamond 9 left, only one card's points need to be deducted, resulting in $R_{a_{\text{Single}(c2)}}^{\text{lose}}(s_3, s_5) = -5$, and the action value $Q_{a_{\text{Single}(c2)}}^{2.0}(s_0, s_i) = -5$. Finally, the action with less loss will be chosen, so player p_0 will play the Club 2.

C. MDP 3.0—Fastest Wining Action Decision

In the design of MDP 1.0 and MDP 2.0, we use scores as rewards. However, in the game of *Big2*, if any player achieves

victory, the game immediately ends, and the scores are settled. Players who have not achieved victory are considered to have lost the game and lose points. Therefore, we aim to find the nearest winning state to reduce the winning possibility of other players. In addition to considering gains and losses, we introduce a separate MDP to specifically account for the distance between the current state and winning state.

In MDP 3.0, we make adjustments to the original action values by multiplying them with corresponding weights based on their distances to winning states. The closer they are to victory, the higher their values, and the farther they are from victory, the lower their values. This approach guides the AI toward faster victory routes while still considering the possibility of achieving high scores through more distant routes.

We adopt a distance variable $d(s')$ that represents how many states are left for state s' to reach the winning state in the shortest path. In addition, $d_{\max}$ is defined as the maximum distance variable, calculated as $d_{\max} = \max d(s')$. The toward-winning value $W(s')$ of state s' is calculated as

$$W(s') = 1 - \frac{d(s')}{d_{\max}} \tag{10}$$

where $W(s')$ is calculated with the action value Q in each round. Therefore, the reward of a new state s' that is closer to victory will have a higher weight, while the value of s' that is farther from victory will be relatively lower. The new action values $\widehat{Q}_a^{1.0}(s, s_i)$ and $\widehat{Q}_a^{2.0}(s, s_i)$ are calculated as

$$\widehat{Q}_a^{1.0}(s, s_i) = \max_{s_i \in C(s)} W(s_i)[P_a^{\text{win}}(s, s_i) \times R_a^{\text{win}}(s, s_i)] \tag{11}$$

and

$$\widehat{Q}_a^{2.0}(s, s_i) = \begin{matrix} \max_{s_i \in C(s)} W(s_i)[P_a^{\text{win}}(s, s_i) \times R_a^{\text{win}}(s, s_i) \\ + P_a^{\text{lose}}(s, s_i) \times R_a^{\text{lose}}(s, s_i)] \end{matrix}, \tag{12}$$

respectively. Therefore, in MDP 3.0, the state value V is calculated as

$$V^{3.0}(s) = \begin{cases} \max_{s_i \in C(s)} \widehat{Q}_a^{2.0}(s, s_i), & \text{if } S_{\text{end}} = \text{True} \\ & \wedge\ \widehat{Q}_a^{2.0}(s, s_i) \le 0 \ \ \forall s_i \\ \max_{s_i \in C(s)} \widehat{Q}_a^{1.0}(s, s_i), & \text{otherwise.} \end{cases} \tag{13}$$

Fig. 4 shows an example of the *Big2* game in MDP 3.0, where the distance variable $d(s')$ is indicated by a pentagon. States s_7, s_8, and s_9 are just one state away from victory, so their distance variables are the same (i.e., $d(s_7) = d(s_8) = d(s_9) = 1$). State s_4 needs to go through state s_8 to achieve victory, so its distance variable is $d(s_4) = 2$. State s_5 can only lose the game, so it has no distance variable. State s_1 needs to go through states s_4 and s_8 to achieve victory, so its distance variable is $d(s_1) = 3$. State s_3 has two winning paths: going through state s_6 requires 3 steps, while going through state s_7 only requires 2 steps. The distance variable will take the shortest path, so $d(s_3) = 2$. The maximum distance in Fig. 4 is $d_{\max} = 3$. The toward-winning values are calculated as follows: $W(s_8) = 1 - 1/3 = 2/3$, $W(s_6) = 1 - 2/3 = 1/3$, and $W(s_1) = 1 - 3/3 = 0$. Suppose that states s_7 and s_9 have the same reward. If player p_0 is currently in state s_3, then because the distance variables $d(s_6)$ and $d(s_7)$ of states s_6 and s_7 are 2 and 1, respectively, player p_0 would tend to choose action a_6, which leads to state s_7.

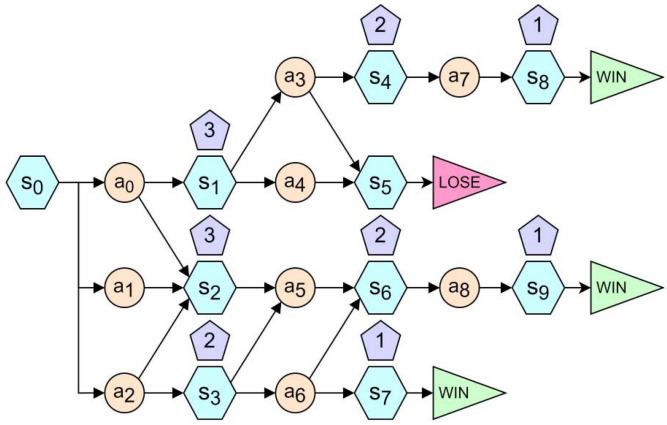


Fig. 4. Example of fastest winning action in MDP 3.0.

D. MDP 4.0—Highest Wining Rate Decision

In MDP 1.0, we calculate the maximum score each route could achieve. In MDP 2.0, we address the potential score losses. In MDP 3.0, we use weights to guide the AI toward faster winning paths. However, in *Big2*, any route can be intercepted by opponents, and if they play a card combination that cannot be surpassed, our predicted card-playing route may be disrupted. One solution is to compete for the right to change card combinations (i.e., free-playing right) which allows us to play any card combination, where there is no chance to block our next move. Therefore, on the basis of MDP 1.0, 2.0, and 3.0, the goal is to obtain the free-playing right and using it to select the optimal card-playing path in MDP 4.0.

In the *Big2* game, a player must play a card combination of the same type as the one played by the previous player. If the other three opponents choose to pass after a player plays a card set, the player gains the right to change card combinations and is no longer required to play a card combination of the same type as the previous one. The player can freely play cards that comply with the rules of the card type. Therefore, obtaining the free-playing right allows a player to play low-ranking cards that are difficult to play out or high-ranking cards that make it challenging for opponents to play cards for continuously gaining the free-playing right. However, on the contrary, if a player prematurely plays high-ranking cards to gain the free-playing right but fails to do so, they may be left with only low-ranking cards and lose the opportunity to win the game. Hence, one must carefully choose the timing to gain the free-playing right. For MDP 4.0, it is allowed to flexibly decide which MDP to be performed [37] and designed with four strategies 1) win-rate prioritizing (WP), 2) movement predicting (MP), 3) weak covering (WC), and 4) series preempting (SP) in the following sections, where the latter strategies are based on the former strategies.

1) Win-Rate Prioritizing: To avoid having the winning route intercepted, we design a decision-making process to achieve the highest winning probability. This involves calculating the

winning probability for each card-playing route and finally selecting the one with the highest winning probability, which means it is the least likely to be intercepted by opponents. This decision-making process simultaneously modifies the values for both aggressive and conservative actions. In the previous action values, the scoring rewards tend to make the MDP choose high-risk routes, which are more likely to be intercepted by opponents. If a player tries to achieve a high score by playing high-ranking cards and ends up getting intercepted, which is unable to play low-ranking cards from his/her hand, it may lead to even more points being lost. On the other hand, the weighting for the fastest winning favors preserving and playing card combinations with more cards. However, in situations where a player does not have the free-playing right, if none of the opponents play card combinations with more cards, it could result in the player being left with many cards and losing the game.

Therefore, in the action values related to winning, we remove the rewards for the fastest winning and scoring, and solely use the probability of the action leading to a player's own victory state as

$$Q_a^{\text{win}}(s, s_i) = \max_{s_i \in C(s)} P_a^{\text{win}}(s, s_i). \tag{14}$$

Similarly, the conservative action values have to be modified, just like the reason for removing the fastest winning weighting from the winning action values, which lead to the route with the least points lost. Therefore, in the conservative action values, we remove all probabilities and rewards related to winning, and only consider the probabilities of failure and the points lost in case of failure as

$$Q_a^{\text{lose}}(s, s_i) = \max_{s_i \in C(s)} P_a^{\text{lose}}(s, s_i) \times R_a^{\text{lose}}(s, s_i). \tag{15}$$

Finally, in MDP 4.0, the state value V is calculated as

$$V_{\text{win}}^{4.0}(s) = \begin{cases} \max_{s_i \in C(s)} Q_a^{\text{loss}}(s, s_i), & \text{if } S_{\text{end}} = \text{True} \\ & \wedge\, Q_a^{\text{win}}(s, s_i) \le 0 \quad \forall s_i \\ \max_{s_i \in C(s)} Q_a^{\text{win}}(s, s_i), & \text{otherwise.} \end{cases} \tag{16}$$

2) Movement Predicting: In opponents' movement prediction, we remove all the played cards C_{played} on the table and the cards C_p in the hand of player p_0 from the total deck C_{all} to obtain the remaining cards C_{rest} yet to be played. We define the card set played by player p_0 as the first played card set for each state. We construct the prediction tree based on the remaining cards C_{rest}. In a real card game, these remaining cards C_{rest} are unevenly distributed among the three opponents p_1, p_2, and p_3, and their card plays are influenced by the cards they hold. If the AI could not determine how the remaining cards are distributed among the three opponents' hands, as a result, the calculated results may differ from the real situation, leading to missed opportunities for potential victories. For example, if player p_1's hand contains the Diamond 2 and player p_1 also hold the Spade 2, player p_1 can confidently play the Diamond 2 as a single card to secure the right to play next. However, if player p_2 holds the Spade 2, player p_1 may choose not to compete for the free-playing right, as there is a chance that player p_2 could play the Spade 2 to beat the Diamond 2 played by player p_1.

In order to predict the possible card play routes of the opponents, additional information has to be added in the original state. We add a number of features to the states, and each state records its features based on historical game data in the training process. Four features are recorded: 1) the first played card set in the state, 2) the Table-Card Level (i.e., T) of the state, 3) the number of remaining cards for each of the three opponents, and 4) the number of remaining cards in p_0's hand. When constructing the prediction tree, we compare the features of historical states with the real card play in the current game round. If at least one of the four features matches the feature of the current state, the state will be selected into the prediction tree. In particular, if multiple features are required to be matched, it may result in the prediction tree with insufficient selected states. States with no matching feature will be removed. The state that meets this condition is denoted as $S_f = \text{True}$ as follows, where $F(s)$ is the feature set of state s. Note that the symbols $F^a(s)$, $F^T(s)$, and $F^h(s)$ are abbreviated as F^a, F^T, and F^h, respectively, and the features of a_0, T, h_0, h_1, h_2, and h_3 belong to the state s:

$$S_f = \begin{cases} \text{True}, & \text{if } a_0 \in F^a \vee l \in F^T \vee |h_0| \in F_0^h \\ & \vee\, (|h_1| \in F_1^h \wedge |h_2| \in F_2^h \wedge |h_3| \in F_3^h) \\ \text{False}, & \text{otherwise} \end{cases} \tag{17}$$

where a_0 is the first played card set in the state s, F^a is the feature set of the first played card sets in the state s, l is the Table-Card Level that is the sum of scores of cards played on the table in state s, F^T is the feature set of Table-Card Levels in state s, h_i is the hand cards of player p_i, and $|h_i|$ is the number of cards in h_i's hand, F_i^h is the feature set of the numbers of cards in player p_i's hand in state s. States that are selected have their transition probabilities adjusted based on their occurrence frequencies as

$$P_a^{MP}(s, s') = \frac{N(s, a, s')}{N(s, s')}, \quad \text{for } s' \in C_f(s) \tag{18}$$

where the new state s' belongs to the state set $C_f(s)$, and $C_f(s)$ is the set of new states that satisfy the condition $S_f = \text{Ture}$ in state s.

3) Weak Covering: A common method to gain the free-playing right is using high-ranking cards to obtain the right to change card combinations and then playing difficult-to-play low-ranking cards. We calculate the probability of obtaining the free-playing right for each action as a reward R^c. For action a, the probability of obtaining the free-playing right in the new state s' when transitioning from state s to s' is determined by whether all opponent actions are a_{Pass}

$$R_a^c(s, s') = \frac{K_1(s') + K_2(s') + K_3(s')}{3} \tag{19}$$

where

$$K_i(s') = \begin{cases} 1, & \text{if } a_i = a_{\text{Pass}}, \text{for } a_i \in M(s_i) \\ 0, & \text{otherwise} \end{cases} \tag{20}$$

where a_i is the action of player p_i in state s_i and $a_i = a_{\text{Pass}}$ indicates that player p_i's action is not to play any card.

The state value for the strategy of high-ranking cards covering low-ranking cards is the percentage of states that satisfy the

condition S_{cover} and the probability of gaining the free-playing right

$$V^{4.0}_{\text{cover}}(s,a) = \sum_{i=1}^{|C_{\text{cover}}(s)|} R^c_a(s,s_i), \quad \text{for } s_i \in C_{\text{cover}}(s) \tag{21}$$

where $C_{\text{cover}}(s)$ is the set of new states that satisfy the condition S_{cover} in the current state s. To determine whether the strategy of high-ranking cards covering low-ranking cards can be executed, MDP 4.0 introduces a new condition S_{cover} including (1) in the current state s. there is at least one card combination with a probability of gaining the free-playing right greater than or equal to $E^{\text{cover}}_{\alpha}$, (2) in the next state s', there is exactly one card combination with a probability of gaining the free-playing right less than or equal to E^{cover}_{β}, and (3) in the subsequent state s'', there is at least one card combination with a probability of gaining the free-playing right greater than or equal to $E^{\text{cover}}_{\gamma}$, where the types of card combinations in s' and s'' are the same and the feasible threshold values of $E^{\text{cover}}_{\alpha}$, E^{cover}_{β}, and $E^{\text{cover}}_{\gamma}$ are obtained based on our experimental results

$$S_{\text{cover}} = \begin{cases} & \text{if } V^{4.0}_{\text{action}}(s,a) \geq E^{\text{cover}}_{\alpha}, \exists\ a \in A_s \\ \text{True}, & \wedge\ V^{4.0}_{\text{action}}(s',a') \leq E^{\text{cover}}_{\beta}, !\exists\ a' \in A_{s'} \\ & \wedge\ V^{4.0}_{\text{action}}(s'',a'') \geq E^{\text{cover}}_{\gamma}, \exists\ a'' \in A_{s''} \\ \text{False}, & \text{otherwise} \end{cases} \tag{22}$$

where

$$V^{4.0}_{\text{action}}(s,a) = \sum_{i=1}^{|C(s,a)|} R^c_a(s,s_i), \text{for } s_i \in C(s,a) \tag{23}$$

where $C(s,a)$ is the set of all possible new states in state s resulting from action a and A_s is the set of all possible actions in state s.

When condition S_{cover} is met, the default state value (i.e., $V^{4.0}_{\text{win}}(s_i)$) will be replaced with a strategy-specific state value (i.e., $V^{4.0}_{\text{cover}}(s_i)$) as

$$V^{4.0}(s) = \begin{cases} \max_{s_i \in C(s)} V^{4.0}_{\text{cover}}(s_i), & \text{if } S_{\text{cover}} = \text{True} \\ \max_{s_i \in C(s)} V^{4.0}_{\text{win}}(s_i), & \text{otherwise.} \end{cases} \tag{24}$$

4) Series Preempting: Another method to compete for the free-playing right is to aim for consecutive card plays, with the goal to hold the free-playing right until the last card is played (i.e., all opponents cannot play any card during consecutive card plays). This approach maximizes the opponent's score reduction while increasing the winner's own score. As gaining the free-playing right requires playing high-ranking cards to prevent opponents from surpassing, it is crucial to calculate the expected probability of gaining the free-playing right for each action in every possible turn to decide whether to execute this strategy. In addition, since the last card set will be played upon obtaining the free-playing right, it is sufficient to have $i-1$ card sets with high probabilities of gaining the free-playing right, where i is the total number of card sets in hand.

In MDP 4.0, the state value for a series of free-playing rights is adopted for consecutive card plays

$$V^{4.0}_{\text{series}}(s) = \sum_{i=1}^{|C_{\text{series}}(s)|} R^c_a(s,s_i), \quad \text{for } s_i \in C_{\text{series}}(s) \tag{25}$$

where $C_{\text{series}}(s)$ is the set of new states that satisfy the condition S_{series} in the current state s. The condition S_{series} is defined as follows, where there is only one or zero card set with a probability of gaining the free-playing right less than or equal to $E^{\text{series}}_{\beta}$, while the probabilities of obtaining the free-playing right for the other card sets are greater than or equal to $E^{\text{series}}_{\alpha}$

$$S_{\text{series}} = \begin{cases} & \text{if } |A^H_s| \geq (|A_s|-1) \wedge |A^L_s| \leq 1 \\ \text{True}, & \wedge\ V^{4.0}_{\text{action}}(s,a_h) \leq E^{\text{series}}_{\alpha} \quad \forall\ a_h \in A^H_s \\ & \wedge\ V^{4.0}_{\text{action}}(s,a_l) \geq E^{\text{series}}_{\beta} \quad \forall\ a_l \in A^L_s \\ \text{False}, & \text{otherwise} \end{cases} \tag{26}$$

where A^H_s and A^L_s are the sets of all possible actions in state s with high and low probabilities of gaining the free-playing right, respectively, and $|A^H_s|$ and $|A^L_s|$ are the numbers of actions in the sets A^H_s and A^L_s, respectively. Similarly, when condition S_{series} is met, the default state value will be replaced with a strategy-specific state value as

$$V^{4.0}(s) = \begin{cases} \max_{s_i \in C(s)} V^{4.0}_{\text{series}}(s_i), & \text{if } S_{\text{series}} = \text{True} \\ \max_{s_i \in C(s)} V^{4.0}_{\text{cover}}(s_i), & \text{if } S_{\text{series}} = \text{False} \\ & \wedge\ S_{\text{cover}} = \text{True} \\ \max_{s_i \in C(s)} V^{4.0}_{\text{win}}(s_i), & \text{otherwise.} \end{cases} \tag{27}$$

V. System Implementation

We have developed a Big2MDP system on the Android platform for the four-player *Big2* game. Players can install the client application on their smartphones or tablets and access the game on their mobile devices. The client application connects to our Big2MDP server through the Internet. The server verifies the actions taken by players from their client applications to ensure compliance with the game rules. The Big2MDP server is programmed in C# 4.0 and utilizes the. NET version 4.6.1 framework and its underlying libraries. It employs MySQL version 8.0.12 as the database for storage. Data transmission between the server and clients is achieved using Google Protocol Buffers version 3.5.1. After each game ends, the server stores the game records for subsequent AI learning purposes. However, during an ongoing game, the server does not continuously save game data. Instead, it only validates and communicates actions taken by all players to conserve client-side data transmission and server resources. When it is the AI's turn to make a move, the necessary game information required for AI computations is sent to the Big2MDP server by the previous player.

Fig. 5(a) shows the game table lobby, where players can create a new game session and invite either AI players or real players to join. Each game session can accommodate up to four players. During game setup, players have the option to open the AI difficulty menu. AI players can choose from four different difficulty levels (i.e., Rookie, Normal, Expert, and Master), corresponding to MDP 1.0, MDP 2.0, MDP 3.0, and MDP 4.0, as designed in our framework. To prevent excessive waiting times for other

(a) (b) (c)

Fig. 5. (a) Game table lobby, (b) in-game interface, and (c) end-of-game screen of the Big2MDP system.

players, there is a 10-s time limit for making moves during the game. Typically, Big2MDP can make decisions within this time limit. In case a real player exceeds the allotted time without taking action in his/her turn, the game will automatically allow the basic AI, MDP 1.0, to take over the remaining gameplay. Alternatively, players can manually click the Take Over button to let the AI handle the game. At any point, players can click the Stop Take Over button to resume playing the game themselves.

Fig. 5(b) displays the in-game interface, which differs from the typical top–down view commonly seen in poker games. Instead, we adopt a first-person perspective to simulate a realistic gameplay experience. According to the rules of the game, players cannot view each other's hands, ensuring fairness in the gameplay even for AI players. At the beginning of each game session, all real players deduct an entry fee to start with an initial chip stack. When the game session ends, all players' remaining cards are revealed, and their gains and losses are calculated. If any player's chip count falls below zero or the maximum number of games is reached, the game session is closed, and the final scores are tallied and reflected in the players' overall scores. In our game rules, each card is worth 5 points. The remaining card with the number 2 makes total losing points doubled, and if a player holds equal to or more than 10 cards, the total losing points are also doubled. Fig. 5(c) shows the end-of-game screen, where players 2, 3, and 4 have lost this game and are penalized 10 points (for two cards without number 2), 200 points (for ten cards with number 2), and 30 points (for three cards with number 2), respectively. On the other hand, player 1 has won this game and gained the total of 240 points, which are deducted from the other players.

VI. Performance Evaluation

In this section, we conduct a series of performance evaluations. First, we test and select the feasible threshold values for Big2MDP parameters. Next, we verify the learning effectiveness of different MDP AI versions and assess the impacts of card-playing route prediction and free-playing right competition on gameplay in MDP 4.0. Finally, we compare our designed AI with existing *Big2* AIs from literature works [34], [35], [36] and analyze the improvement in win rates and scores with varying training and playing volumes. The experiments are conducted on a computer with Intel i5-9500 CPU, operating at a frequency of 3.00 GHz. Table II lists the computation times of MDP AIs with

TABLE II
Computation Times of MDP 1.0, MDP 2.0, MDP 3.0, and MDP 4.0 (ms)

$\|h_0\|$	MDP 1.0	MDP 2.0	MDP 3.0	MDP 4.0
12 – 13	582	956	1235	3218
10 – 11	532	884	1023	2947
8 – 9	483	812	893	2647
6 – 7	443	756	834	2512
4 – 5	353	638	712	2224
1 – 3	251	486	573	1939
Average	432	741	832	2454

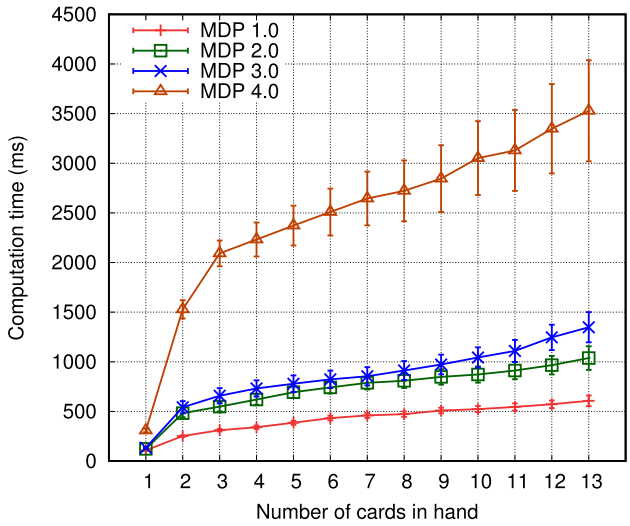


Fig. 6. Computation times with standard deviations for different numbers of cards in hand.

different numbers of hand cards (i.e., $|h_0|$). The computation times of MDP 1.0, MDP 2.0, MDP 3.0, and MDP 4.0 (with all of WP, MP, WC, and SP strategies) are reducing with the decreasing numbers of hand cards and the average times taken are 432, 741, 832, and 2454 ms, respectively. In particular, Fig. 6 shows computation times with standard deviations for different numbers of cards in hand.

First, the experiments focus on the end-game condition, S_{end}, adopted in MDP 2.0, 3.0, and 4.0. This condition is used to determine whether to use the aggressive winning strategy of MDP 1.0 (i.e., $Q^{1.0}$) or the conservative defending strategy of MDP 2.0 (i.e., $Q^{2.0}$). When facing an opponent who is going to win and player p_0 cannot win probably, the decision of whether to play cards that result in higher losses will impact the final

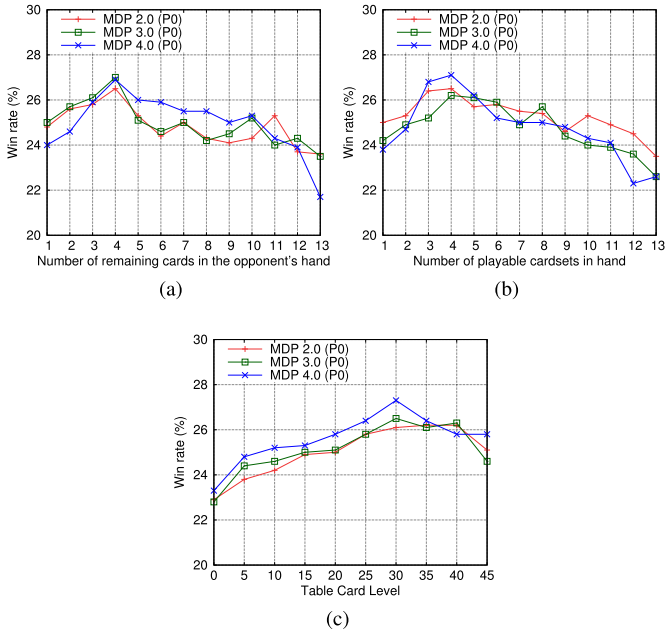


Fig. 7. Win rates for different threshold values (a) E_{α}^{end} , (b) E_{β}^{end} , and (c) E_{γ}^{end} of MDP 2.0, 3.0, and 4.0.

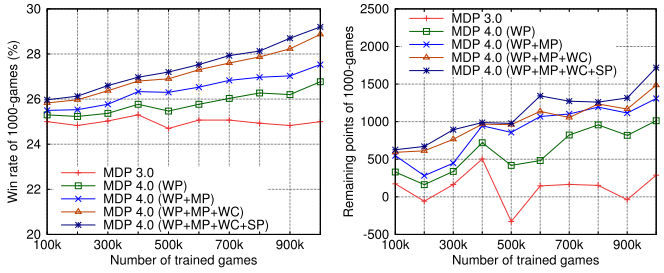


Fig. 8. (a) Win rates and (b) remaining points of MDP 3.0 and MDP 4.0 with different numbers of trained games.

score. On the other hand, constantly holding onto high-ranking cards may cause one to miss the opportunity to win. Hence, the judgment for switching between these two modes must achieve the optimal balance between both strategies.

Fig. 7 shows the win rates for different threshold values of MDP 2.0, 3.0 and 4.0. Each AI is trained for 100 000 games and play 1000 games against three opponents using the fixed threshold values of $E_{\alpha}^{\text{end}} = 3$, $E_{\beta}^{\text{end}} = 3$, and $E_{\gamma}^{\text{end}} = 30$ in S_{end}. In Fig. 7(a), the modified threshold value is based on the remaining numbers of cards in the hands of opponents p_1, p_2, and p_3, denoted as $|h_1| \leq E_{\alpha}^{\text{end}} \vee |h_2| \leq E_{\alpha}^{\text{end}} \vee |h_3| \leq E_{\alpha}^{\text{end}}$. Each version of AI performs best when the remaining card count is between 3 and 5, as too few cards result in a conservative style leading to score loss, while too many cards cause an overly conservative approach that misses opportunities for victory. In Fig. 7(b), the win rates are presented for different threshold values based on the remaining playable card sets in the hand of player p_0, denoted as $|g_0| \leq E_{\beta}^{\text{end}}$. Each version of AI reaches its peak performance when the remaining card sets are around 4, with similar reasoning to the previous experiment. Fig. 7(c) shows the win rates for different threshold values based on the value of the played cards on the table, denoted as $l \geq E_{\gamma}^{\text{end}}$. The highest win rate occurs when the Table-Card Level is 30, and in contrast to the previous two experiments, lower Table-Card Levels indicate the early stage of the game, while higher levels signify a more advanced stage (i.e., closer to the end of the game). Hence, the results exhibit an opposite trend in Fig. 7(a) and (b). Therefore, the threshold values of S_{end} are set to $E_{\alpha}^{\text{end}} = 4$, $E_{\beta}^{\text{end}} = 4$, and $E_{\gamma}^{\text{end}} = 30$ in the following experiments.

Next, we find the feasible trigger threshold value for the strategy of high-ranking cards covering low-ranking cards. The condition $S_{\text{cover}}^{4.0}$ is satisfied under the conditions: $V_{\text{action}}^{4.0}(s,a) \geq E_{\alpha}^{\text{cover}}$, $V_{\text{action}}^{4.0}(s',a') \leq E_{\beta}^{\text{cover}}$, and $V_{\text{action}}^{4.0}(s'',a'') \geq E_{\gamma}^{\text{cover}}$. Unlike the previous experiments, as this trigger involves all three conditions, each experiment will adjust one condition's threshold value while keeping the other two at their default values. Table III lists the win rates for using different threshold values in MDP 4.0. Each AI is trained for 100 000 games, and the average results are taken from 1000 games, with opponents using default threshold values of MDP 4.0 (i.e., $E_{\alpha}^{\text{cover}} = 0.9$, $E_{\beta}^{\text{cover}} = 0.1$, and $E_{\gamma}^{\text{cover}} = 0.9$ in default). The win rates for $V_{\text{action}}^{4.0}(s,a) \geq E_{\alpha}^{\text{cover}}$ and $V_{\text{action}}^{4.0}(s'',a'') \geq E_{\gamma}^{\text{cover}}$ increase with higher threshold values, but when closing to 1.0, they decrease obviously. Conversely, the win rates for $V_{\text{action}}^{4.0}(s',a') \leq E_{\beta}^{\text{cover}}$ increases as the threshold value decreases, but when closing to 0, it also decreases. Therefore, the threshold values of S_{cover} are set to $E_{\alpha}^{\text{cover}} = 0.8$, $E_{\beta}^{\text{cover}} = 0.2$, and $E_{\gamma}^{\text{cover}} = 0.8$ in the following experiments.

Moreover, we find the feasible threshold values to gain the free-playing right for consecutive card plays for the condition S_{series}. Each experiment adjusts one condition's threshold value while keeping the other at its default value. Table IV presents the win rates for this experiment. The average results are taken from 1000 games, with opponents using default threshold values of MDP 4.0 (i.e., $E_{\alpha}^{\text{series}} = 0.9$ and $E_{\beta}^{\text{series}} = 0.1$ in default). In Table IV, the win rate trends for the two conditions are opposite. The win rate for $V_{\text{action}}^{4.0}(s,a) \geq E_{\alpha}^{\text{series}}$ increases as the threshold value rises, while the win rate for $V_{\text{action}}^{4.0}(s,a) \leq E_{\beta}^{\text{series}}$ increases as the threshold value decreases. Both win rates significantly decrease when reaching their respective maximal values. Therefore, the threshold values of $E_{\alpha}^{\text{series}} = 0.8$ and $E_{\beta}^{\text{series}} = 0.1$ are used in the following experiments.

To verify the performance improvement of decision-making strategies designed in MDP 4.0, we conduct gameplays between MDP 3.0 and MDP 4.0 with different strategies including WP, MP, WC, and SP. In each gameplay, the MDP 4.0 faces three instances of MDP 3.0. To ensure fairness comparisons, 1000 games are conducted where we randomly distribute a deck of 52 cards into four groups, with each group receiving 13 cards. We record the distribution of cards in each group for every game and use the same hand-card groups for MDP 3.0 and MDP 4.0 to minimize variations caused by different hand cards.

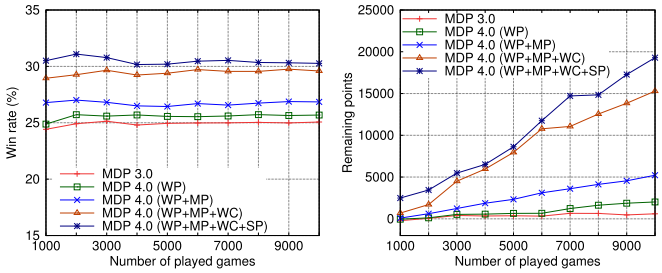
Fig. 8(a) shows the win rates of MDP 3.0 and MDP 4.0 (each faces three instances of MDP 3.0) with different numbers of

TABLE III
WIN RATES USING DIFFERENT THRESHOLD VALUES FOR S_{COVER}

Threshold	0.0	0.1	0.2	0.3	0.4	0.5	0.6	0.7	0.8	0.9	1.0
$E_{\alpha}^{\text{cover}}$	23.0	23.6	23.8	24.2	24.2	24.3	24.4	24.9	25.5	25.1	24.4
E_{β}^{cover}	24.4	25.6	26.0	25.6	24.4	24.7	24.4	24.4	23.6	23.5	23.6
$E_{\gamma}^{\text{cover}}$	23.3	23.2	23.5	24.1	24.6	24.9	25.0	25.1	26.2	25.6	24.6

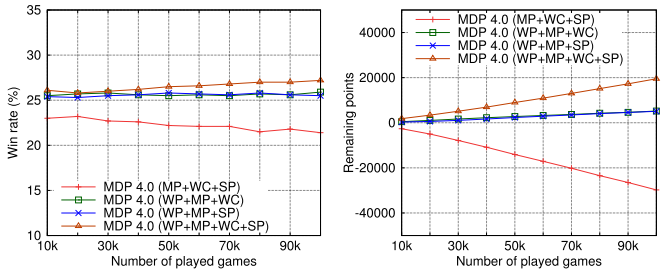
TABLE IV
WIN RATES USING DIFFERENT THRESHOLD VALUES FOR S_{SERIES}

Threshold	0.0	0.1	0.2	0.3	0.4	0.5	0.6	0.7	0.8	0.9	1.0
$E_{\alpha}^{\text{series}}$	22.6	23.3	23.3	23.8	24.3	24.3	24.5	25.3	26.2	25.7	24.6
$E_{\beta}^{\text{series}}$	23.5	25.8	25.4	25.5	25.1	24.0	24.1	24.1	23.7	23.3	22.8



(a) (b)

Fig. 9. (a) Cumulative win rates and (b) remaining points of MDP 3.0 and MDP 4.0 with different numbers of played games.

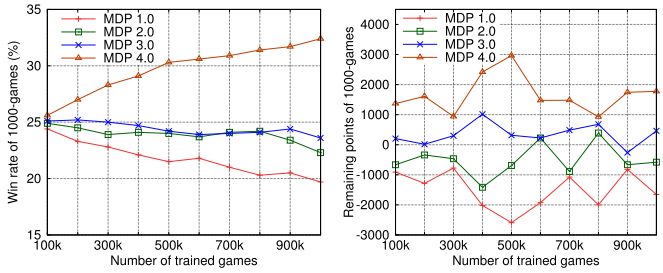


(a) (b)

Fig. 10. (a) Cumulative win rates and (b) remaining points of independent agents for MDP 4.0 with different numbers of played games.

trained games. The win rates of MDP 3.0 remain around 25%, indicating that the hand cards had little impact on the results. In contrast, the designed strategies of WP, MP, and WC in MDP 4.0 led to a significant improvement in win rates under the same hand-card groups, while the effect of SP on win rates is minor but the remaining points are increasing. Fig. 8(b) shows the remaining points of MDP 3.0 and MDP 4.0 in the games. Both MDP 3.0 and MDP 4.0 with different strategies exhibit changes in scores with the increased number of trained games. Although SP has a slight impact on win rates, it significantly affects the scores. This is because SP helps MDP to acquire more points as hands with high win rates remain victorious. Consequently, the competition of consecutive card plays in SP reduces the number of cards opponents can play, leading to higher final scores for the AI utilizing this strategy. Fig. 9(a) shows the cumulative win rates (facing three instances of MDP 3.0), while Fig. 9(b) shows the cumulative scores, where MDP 4.0 with WP, MP, WC, and SP significantly outperforms the other AIs in terms of both wins and scores. In particular, Fig. 10 shows an ablation study consisting of independent agents (each with one of the strategies removed) for MDP 4.0 to determine the impact of each strategy. It can be seen that WC and SP have the similar performance (both affected by the luckiness of high-ranking cards in dealing cards), whereas WP has the most significant improvement (through avoiding the winning route being intercepted).

Fig. 11 shows the win rates and remaining points of MDP 1.0, MDP 2.0, MDP 3.0, and MDP 4.0 with different numbers of trained games. In the experiments, the four AIs play 1000 games together, and we record their victories and scores. Fig. 11(a) shows that MDP 1.0 consistently perform the worst



(a) (b)

Fig. 11. (a) Win rates and (b) remaining points of MDP 1.0, MDP 2.0, MDP 3.0, and MDP 4.0 with different numbers of trained games.

from the beginning, while MDP 2.0 and MDP 3.0 rank third and second, respectively. As playing against each other, MDP 4.0 outperforms the other three AIs, achieving the highest win rate. Fig. 11(b) shows the remaining points of each 1000 games. Similarly, MDP 4.0 achieves the highest score throughout the experiment, outperforming the other three AIs. This shows that the designed strategies in MDP 4.0 provide significant advantages in achieving better performance through predicting opponents' movements and competing for free-playing rights. As shown in Fig. 12, similar results are achieved for 2-to-2 performances of MDP 3.0 and MDP 4.0 with different numbers of played games, where MDP 4.0 AIs have much higher win rates and remaining points than MDP 3.0 AIs with the increasing number of played games.

Next, we compare our Big2MDP with existing AI agents, Rule-based AI [34], RL-PPO [35], and Big2AI 4.0 [36], playing

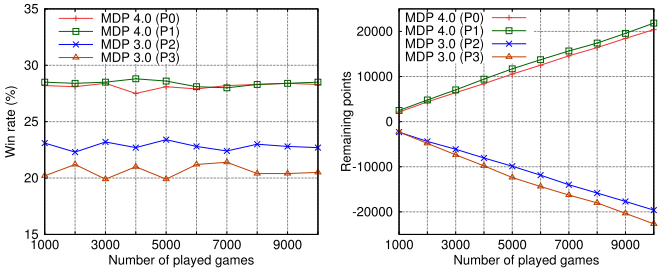


Fig. 12. (a) Cumulative win rates and (b) remaining points of two MDP 3.0 and two MDP 4.0 AIs with different numbers of played games.

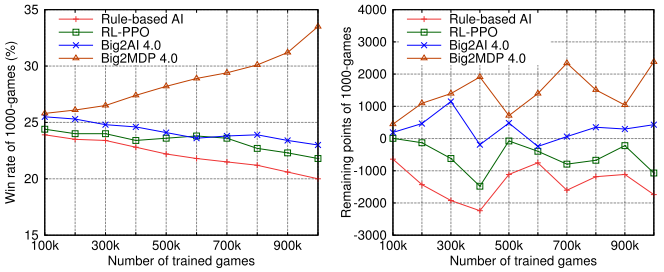


Fig. 13. (a) Win rates and (b) remaining points of Big2MDP 4.0, Rule-based AI, RL-PPO, and Big2AI 4.0 with different numbers of trained games.

against each other in the *Big2* game to validate our improvement. Rule-based AI follows predefined strategies for card playing, while RL-PPO utilizes reinforcement learning with known information to obtain the optimal card-playing strategy. Big2AI 4.0 leverages available information to search past game records for predicting opponents' card plays and calculating the most suitable card-playing actions.

Fig. 13 shows the results of Big2MDP 4.0 and the three aforementioned Big2 AI agents, where rule-based AI does not have the self-learning ability and is not refined by the increasing number of trained games. Fig. 13(a) shows the win rates of the four AI agents, ranked from low to high as follows: rule-based AI, RL-PPO, Big2AI 4.0, and Big2MDP 4.0. This is because Big2MDP 4.0 considers high-chance-to-win routes and predicts opponents' actions to optimize our decision-making, which can converge to the better optimum. Fig. 13(b) presents the remaining points of each 1000 games, with rankings corresponding to their win rates. This is because Big2MDP 4.0 competes for the free-playing right to cover low-ranking cards and/or play high-ranking cards to prevent opponents from surpassing at the essential moment. Fig. 14 shows the cumulative win rates and remaining points with a fixed training volume (i.e., 500 K games) for the four AI agents. Fig. 14(a) shows the cumulative win rates, while Fig. 14(b) shows the cumulative scores, where Big2MDP 4.0 significantly outperforms the other three AI agents in terms of both wins and scores. This because Big2MDP 4.0 simultaneously considers scoring and losing points to make the best winning decisions with minimal losing risk. Fig. 15 shows the win rates and remaining points of two Big2MDP 4.0 and two

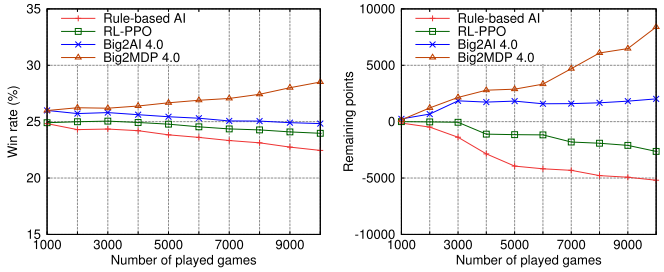


Fig. 14. (a) Cumulative win rates and (b) remaining points of Big2MDP 4.0, Rule-based AI, RL-PPO, and Big2AI 4.0 with different numbers of played games.

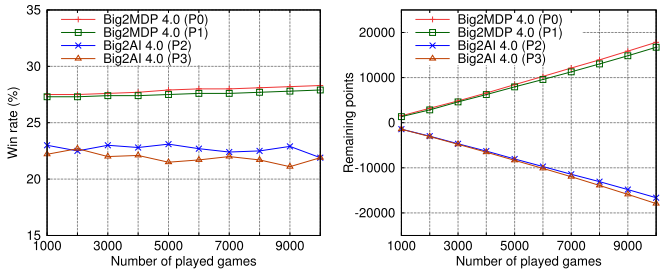


Fig. 15. (a) Cumulative win rates and (b) remaining points of two Big2MDP 4.0 and two Big2AI 4.0 AIs with different numbers of played games.

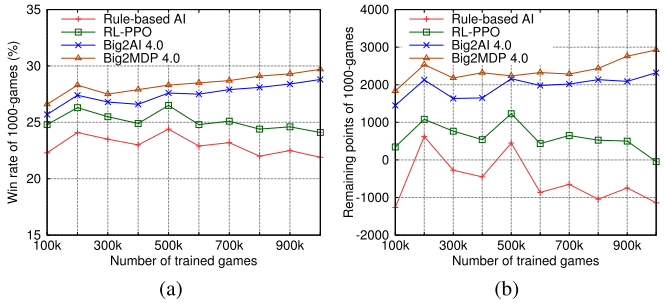


Fig. 16. (a) Win rates and (b) remaining points of Big2MDP 4.0, Rule-based AI, RL-PPO, and Big2AI 4.0 with the same hand card groups.

Big2AI 4.0 AIs with different numbers of played games, where Big2MDP 4.0 significantly outperforms Big2AI 4.0.

Fig. 16 shows the results of game plays for Big2MDP 4.0, rule-based AI, RL-PPO, and Big2AI 4.0 with the same hand-card groups, where the opponents are three Big2MDP 3.0 players. In Fig. 16(a), the win rates of the four AIs are shown, similarly, ranked from lowest to highest as follows: rule-based AI, RL-PPO, Big2AI 4.0, and Big2MDP 4.0. Fig. 16(b) shows the total scores of the four AIs, with Big2AI 4.0 and Big2MDP 4.0 exhibiting rising similar to their win rates. Fig. 17 shows cumulative win rates and remaining points using the same hand-card groups with fixed training volume (i.e., 500 K games). In Fig. 17(a), the cumulative number of wins is shown, and Big2AI 4.0 and Big2MDP 4.0 demonstrate higher win rates compared to the other two AIs. Fig. 17(b) shows the cumulative scores,

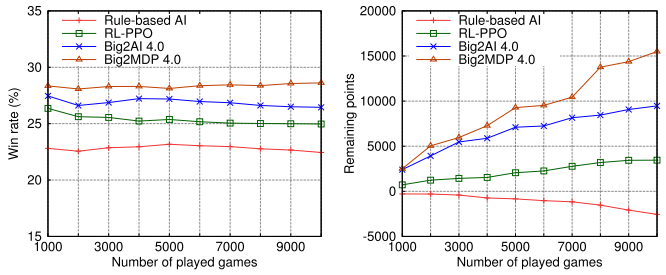


Fig. 17. (a) Cumulative win rates and (b) remaining points of Big2MDP 4.0, Rule-based AI, RL-PPO, and Big2AI 4.0 with the same hand card groups.

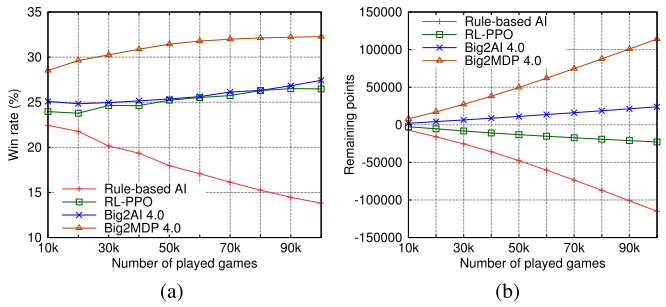


Fig. 18. (a) Cumulative win rates and (b) remaining points of Big2MDP 4.0, Rule-based AI, RL-PPO, and Big2AI 4.0 for playing 10K, 20K, . . ., and 100K games.

where Big2MDP 4.0 attains higher scores than Big2AI 4.0. In particular, with more played games, Big2MDP 4.0 achieves much higher win rates and remaining points than rule-based AI, RL-PPO, and Big2AI 4.0, as shown in Fig. 18.

Finally, we had Big2MDP 4.0 play online *Big2* games and compete against other human players in the *Game Sofa Big Two* [38], where the human players are automatically matched. Big2MDP 4.0 learns in advance with a fixed number of 500 K games. Each game session involves playing four games with three randomly matched opponents, and the scores are calculated at the end of each session. In total, 25 game sessions were played, resulting in a total of 100 games. For a four-player card game, the win rate of an ordinary player in average is about 25%. Therefore, the human opponents are classified based on their historical win rates into three groups: master group, with win rates above 26%, comprising 7 players; Experienced group, with win rates between 26% and 24%, comprising 59 players; and Novice group, with win rates below 24%, comprising 9 players. In particular, all human players in each group have played more than 100 games before our evaluation, where new players are not included in the three groups.

Table V lists the gaming data of Big2MDP 4.0 and the opponents it faced. Since each game session might not necessarily include players from all three groups, the win rates for each group are calculated individually as the number of wins divided by the total games played. Among the groups, the Master group has the highest win rate of 32.14% with an average score of 1.43 points per game. The Experienced group follows with the second-highest win rate of 17.37% and an average score of -1.54 points per game. The Novice group has a win rate of 13.89% and an average score of -1.8 points per game. In comparison, Big2MDP 4.0 achieves an outstanding win rate of 45% and an impressive average score of 3.88 points per game, which significantly outperforms the human players. Note that Big2MDP 4.0 has the similar performance to the master group where there is at least one master player in the game. This is because only 7 master players are assigned to the game sessions (i.e., 28 games) by the *Game Sofa Big Two* platform, where the effects of luckiness in dealing cards cannot be ignored for Big2MDP 4.0 and master players. In particular, the hand with more high-ranking cards has much higher chance to win than that with more low-ranking cards for expert players.

TABLE V
PERFORMANCE OF BIG2MDP 4.0 AGAINST HUMAN PLAYERS

Historical Win Rate	Big2MDP 4.0	Master Group $\geq 26\%$	Experienced Group $26\% \sim 24\%$	Novice Group $\leq 24\%$
Number of Players	1	7	59	9
Games Played	100	28	236	36
Games Won	45	9	41	5
Win Rate	45.00%	32.14%	17.37%	13.89%
Final Score	388	40	-363	-65
Average Score	3.88	1.43	-1.54	-1.80
Standard Error	1.50	0.68	0.64	1.13

VII. CONCLUSION

In this work, we investigate *Big2* and recent literature on artificial intelligence, comparing the advantages and disadvantages of MDP-based AI methods in the imperfect information game of *Big2*. Based on MDPs, we design the Big2MDP artificial intelligence framework and implement it as an Android application for experimental evaluation. The Big2MDP AI combines several MDPs with different rewards, including rewards for seeking the highest score, reducing losses, and achieving the fastest victory. In addition, it utilizes feature extraction of game rounds to predict opponents' movements, assisting in constructing possible states and actions. Furthermore, it strategically contends for the free-playing right for covering low-ranking cards and/or performing consecutive card plays in the *Big2* game. Our Big2MDP outperforms existing *Big2* AIs, achieving higher win rates and remaining points, as well as attaining outstanding win rates and scores in online games against human players. For future works, we are exploring transformer-based reinforcement learning to accurately predict the possible card-playing routes of the opponents for further improving win rates and lost points.

REFERENCES

[1] "Card game big two," 2001. [Online]. Available: https://www.pagat.com/climbing/bigtwo.html
[2] D. Whitehouse, E. J. Powley, and P. I. Cowling, "Determinization and information set Monte Carlo tree search for the card game Dou Di Zhu," in *Proc. IEEE Conf. Comput. Intell. Games (CIG)*, 2011, pp. 87–94.
[3] C. B. Browne et al., "A survey of Monte Carlo tree search methods," *IEEE Trans. Comput. Intell. AI Games*, vol. 4, no. 1, pp 1–43, Mar. 2012.
[4] E. Steinmetz and M. Gini, "More trees or larger trees: Parallelizing Monte Carlo tree search," *IEEE Trans. Games*, vol. 13, no. 3, pp 315–320, Sep. 2021.

[5] J. P. A. M. Nijssen and M. H. M. Winands, "Monte-Carlo tree search for the game of Scotland yard," in *Proc. IEEE Conf. Comput. Intell. Games (CIG)*, 2011, pp. 158–165.
[6] N. Mizukami and Y. Tsuruoka, "Building a computer Mahjong player based on Monte Carlo simulation and opponent models," in *Proc. IEEE Conf. Comput. Intell. Games (CIG)*, 2015, pp. 275–283.
[7] G. Tan, Y. He, H. Xu, P. Wei, P. Yi, and X. Shi, "Winning rate prediction model based on Monte Carlo tree search for computer Dou Dizhu," *IEEE Trans. Games*, vol. 13, no. 2, pp 123–137, Jun. 2021.
[8] P. I. Cowling, E. J. Powley, and D. Whitehouse, "Information set Monte Carlo tree search," *IEEE Trans. Comput. Intell. AI Games*, vol. 4, no. 2, pp 120–143, Jun. 2012.
[9] S. Gao and S. Li, "A review of incomplete information game about games," in *Proc. Int. Conf. Mach. Learn., Big Data Bus. Intell.*, 2019, pp. 36–42.
[10] M. Xian et al., "Research on computer game of incomplete information," in *Proc. Chin. Control Decis. Conf.*, 2015, pp. 5807–5810.
[11] W. Zha, J. Chen, and Z. Peng, "Dynamic multi-team antagonistic games model with incomplete information and its application to multi-UAV," *IEEE/CAA J. Automatica Sinica*, vol. 2, no. 1, pp 74–84, Jan. 2015.
[12] D. Zhu, Y. Zheng, Q. Ren, and P. Sun, "Differential game guidance law for 3-party game with incomplete information," in *Proc. China Autom. Congr.*, 2021, pp. 2003–2008.
[13] J. Chen and Y. Huang, "Decision tree construction algorithm for incomplete information system," in *Proc. Int. Conf. Comput. Inf. Sci.*, 2012, pp. 404–407.
[14] M. Zinkevich, M. Johanson, M. Bowling, and C. Piccione, "Regret minimization in games with incomplete information," in *Proc. Int. Conf. Neural Inf. Process. Syst.*, 2007, pp. 1729–1736.
[15] P. Cotae and N. E. A. Reindorf, "Using counterfactual regret minimization and Monte Carlo tree search for cybersecurity threats," in *Proc. IEEE Int. Black Sea Conf. Commun. Netw. (BlackSeaCom)*, 2021, pp. 1–6.
[16] J. Yao, Z. Zhang, L. Xia, J. Yang, and Q. Zhao, "Solving imperfect information poker games using Monte Carlo search and POMDP models," in *Proc. IEEE Data Driven Control Learn. Syst. Conf. (DDCLS)*, 2020, pp. 1060–1065.
[17] D. Silver, J. Schrittwieser, K. Simonyan, and I. Antonoglou, "Mastering the game of Go without human knowledge," *Nature*, vol. 550, no. 7676, pp 354–359, Oct. 2017.
[18] H. N. Ward, B. Mills, D. J. Brooks, D. Troha, and A. S. Khakhalin, "AI solutions for drafting in magic: The gathering," in *Proc. IEEE Conf. Games (CoG)*, 2021, pp. 1–8.
[19] J. S. B. Choe and J.-K. Kim, "Enhancing Monte Carlo tree search for playing Hearthstone," in *Proc. IEEE Conf. Games (CoG)*, Aug. 2019, pp. 1–7.
[20] L. Critch and D. Churchill, "Sneak-attacks in StarCraft using influence maps with heuristic search," in *Proc. IEEE Conf. Games (CoG)*, 2021, pp. 1–8.
[21] B. Bouzy, A. Rimbaud, and V. Ventos, "Recursive Monte Carlo search for bridge card play," in *Proc. IEEE Conf. Games (CoG)*, 2020, pp. 229–236.
[22] N. Brown and T. Sandholm, "Superhuman AI for heads-up no-limit poker: Libratus beats top professionals," *Science*, vol. 359, no. 6374, pp 418–424, Jan. 2018.
[23] D.-W. Kim, S. Park, and S.-I. Yang, "Mastering fighting game using deep reinforcement learning with self-play," in *Proc. IEEE Conf. Games (CoG)*, 2020, pp. 576–583.
[24] J. Perolat et al., "Mastering the game of Stratego with model-free multiagent reinforcement learning," *Science*, vol. 378, no. 6623, pp 990–996, Dec. 2022.
[25] Y. Zhou, "From AlphaGo zero to 2048," in *Proc. Workshop Deep Reinforcement Learn. Knowl. Discov. (DRL4KDD)*, 2019, pp. 1–7.
[26] M. N. Moghadasi, A. T. Haghighat, and S. S. Ghidary, "Evaluating Markov decision process as a model for decision making under uncertainty environment," in *Proc. Int. Conf. Mach. Learn. Cybern. (ICMLC)*, 2007, pp. 2446–2450.
[27] B. Wu and Y. Feng, "Policy reuse for learning and planning in partially observable Markov decision processes," in *Proc. 4th Int. Conf. Inf. Sci. Control Eng.*, 2017, pp. 549–552.
[28] M. d. G. Garcia-Hernandez, J. Ruiz-Pinales, A. Reyes-Ballesteros, E. Onaindia, and J. G. Avina-Cervantes, "Reducing computational complexity in Markov decision processes using abstract actions," in *Proc. 7th Mex. Int. Conf. Artif. Intell.*, Oct. 2008, pp. 256–260.
[29] C. Georgaraki, "A POMDP approach to the Hide and Seek game," Master Thesis, Universitat Politecnica de Catalunya, Barcelona, Spain, Feb. 2012.
[30] M. Kurita and K. Hoki, "Method for constructing artificial intelligence player with abstractions to Markov decision processes in multiplayer game of mahjong," *IEEE Trans. Games*, vol. 13, no. 1, pp 99–110, Mar. 2021.
[31] J. Humeau, A. Lebis, M. Vermeulen, and G. Lozenguez, "Planning in the midst of chaos: How a stochastic blood bowl model can help to identify key planning features," in *Proc. IEEE Conf. Games (CoG)*, 2021, pp. 1–4.
[32] R. Uribe, F. Lozano, K. Shibata, and C. Anderson, "Discount and speed/execution tradeoffs in Markov decision process games," in *Proc. IEEE Conf. Comput. Intell. Games (CIG)*, 2011, pp. 79–86.
[33] Y. Zhu and D. Zhao, "Online minimax Q network learning for two-player zero-sum Markov games," *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 33, no. 3, pp 1228–1241, Mar. 2022.
[34] G. S. Fernando and W.-K. Tai, "A rule-based AI method for an agent playing big two," *Appl. Sci.*, vol. 11, no. 9, pp 1–23, May 2021.
[35] H. Charlesworth, "Application of self-play reinforcement learning to a four-player game of imperfect information," 2018, *arXiv:1808.10442*.
[36] L.-W. Chen and Y.-R. Lu, "Challenging artificial intelligence with multi-opponent and multimovement prediction for the card game Big2," *IEEE Access*, vol. 10, pp 40661–40676, 2022.
[37] C. Salge, C. Glackin, and D. Polani, "Empowerment—An introduction," in *Guided Self-Organization: Inception*. Berlin Heidelberg: Springer, 2014, pp. 67–114.
[38] "Game sofa big two," 2008. [Online]. Available: https://www.gamesofa.com/bigtwo/

Lien-Wu Chen (Senior Member, IEEE) received the Ph.D. degree in computer science and information engineering from the National Chiao Tung University, Hsinchu, Taiwan, in 2008.

In 2009, he joined the Department of Computer Science, National Chiao Tung University as Postdoctoral Research Associate, and qualified as Assistant Research Fellow in 2010. From 2012 to 2015, he was an Assistant Professor with the Department of Information Engineering and Computer Science, Feng Chia University, Taichung, Taiwan, where he was the Chief of Operation and Maintenance Section from 2016 to 2018. From 2015 to 2019, he was an Associate Professor, and he is a Full Professor since 2019. He has authored or coauthored 100 journal and conference papers and holds over 20 granted invention patents. His research interests include wireless communication and mobile computing, especially in the Internet of Vehicles, Internet of Things, and Internet of People.

Dr. Chen was the recipient of the more than 30 Best/Excellent Paper Awards, Best/Outstanding Demo Awards, Best Presentation Awards, and Outstanding Paper Publication Awards since 2009, and transferred three technologies to Acer Incorporated in January, June, and November 2011. He is the advisor of over 50 Prototyping Awards, Student Engineering Paper Awards, College Student Research Awards, and Master Thesis Awards since 2014. He is an Honorary Member of Phi Tau Phi Scholastic Honor Society, a Lifetime Member of the Chinese Institute of Electrical Engineering, the Institute of Information and Computing Machinery, and the Taiwan Institute of Electrical and Electronic Engineering, and a Senior Member of ACM.

Yiou-Rwong Lu received the B.S. and M.S. degrees in information engineering and computer science from Feng Chia University, Taichung, Taiwan, in 2021 and 2023, respectively.

His research interests include the Internet of Things and machine learning.

Mr. Lu was the recipient of the Best Paper Award at the Taiwan Academic Network Conference in 2023.